

ENGR-E 533

Deep Learning Systems

Module 02

Deep Neural Network Basics

Minje Kim

Department of Intelligent Systems Engineering

Email: minje@indiana.edu

Website: <http://minjekim.com>

Research Group: <http://saige.sice.indiana.edu>

Meeting Request: <http://doodle.com/minje>



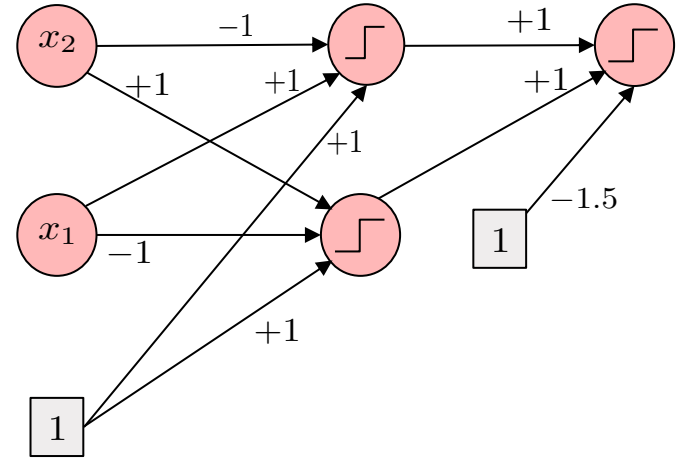
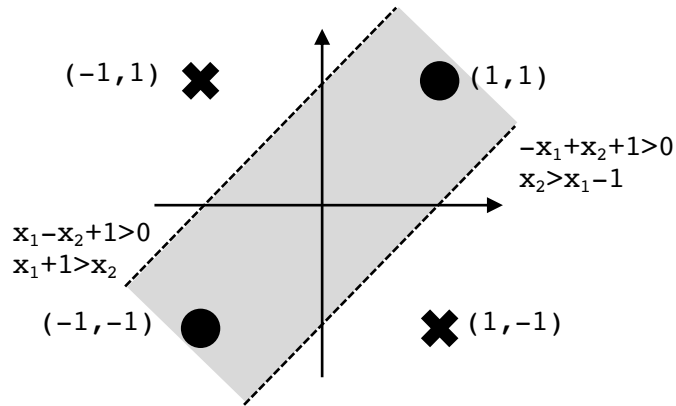
INDIANA UNIVERSITY

**SCHOOL OF INFORMATICS,
COMPUTING, AND ENGINEERING**

Advantage of Deeper Networks

- Parity function

- NN for the XOR problem



- How many parameters are there?



Advantage of Deeper Networks

- Parity function

- A parity function

$$\text{parity}(\mathbf{x}) = \begin{cases} +1 & \text{if the number of } +1 \text{ is odd} \\ -1 & \text{otherwise} \end{cases}$$

- For a vector of bipolar binaries

- For example,

$$\text{parity}([+1, -1, -1, +1, +1, -1]) = +1$$

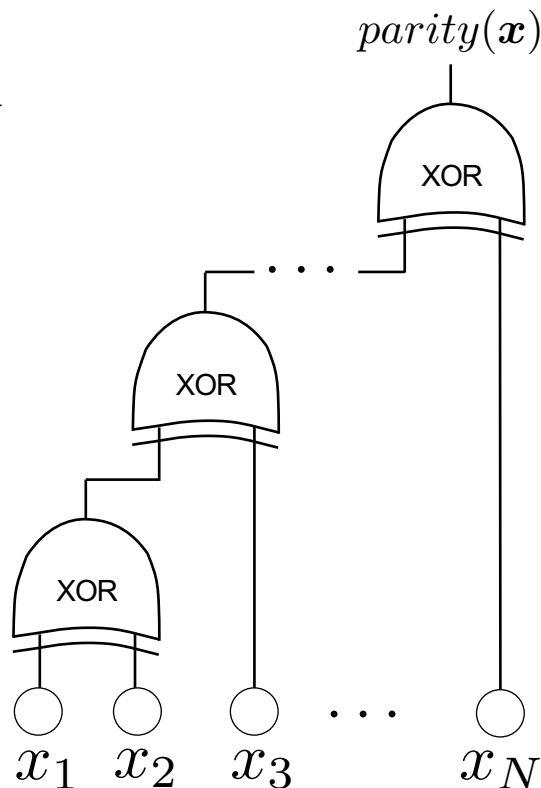
- A network that does this job?

- How many layers?

- N

- How many weights?

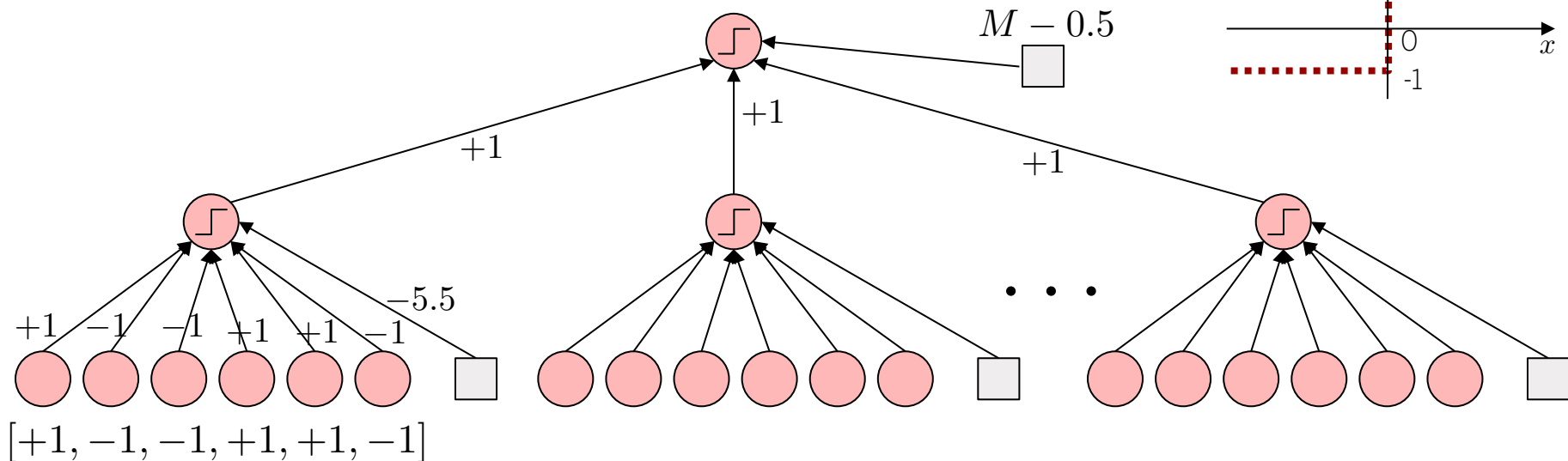
- $9N$



Advantage of Deeper Networks

- Parity function

- A shallower version
- The network can remember all M (unique) input vectors and their parity values



- How many weights do you need?
 - $O(NM)$ weights

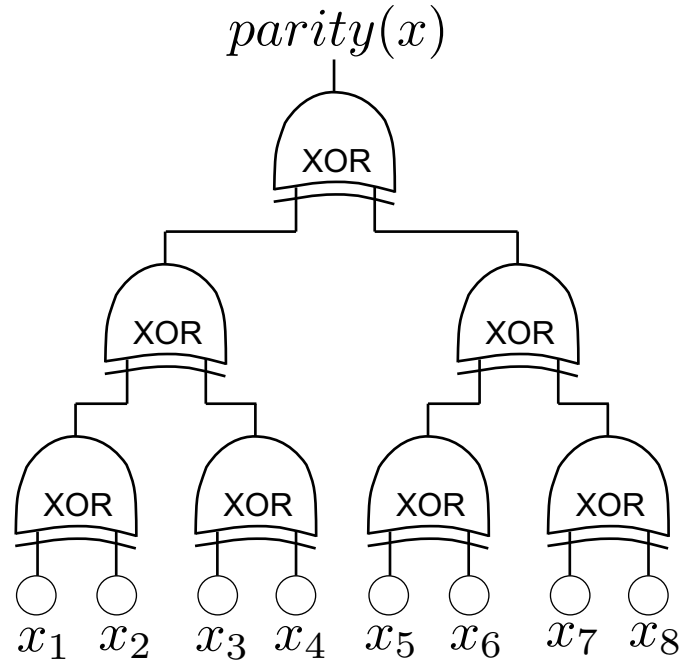


Advantage of Deeper Networks

- Parity function

○ If we reuse the subnetwork carefully

- Depth: $O(\log N)$
- Weights: $O(N)$



Advantage of Deeper Networks

- Summary

- A lot of possible choices for the neural network structure
- In theory (universal approximation theorem)
 - A shallow neural network with one hidden layer can approximate any mapping function
 - Once we're allowed to use as many hidden units as possible
- In practice
 - This shallow and wide network will be with too many weights to estimate
- A Deep Neural Network (DNN)
 - Has more than one hidden layers
 - Is efficient in its structure to learn a very complicated mapping function
- Question:
 - Then, can we just come up with a neural network with lots of layers?
- Answer:
 - Life is not that simple!



Feature Learning for a Shallow Network

- Overview

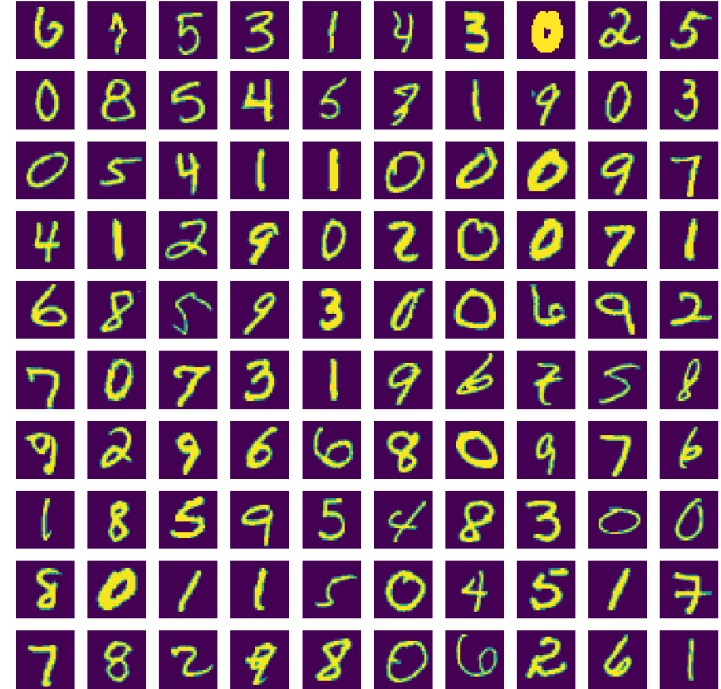
- Feature extraction is important
 - But difficult
 - Usually involves unsupervised learning
- Supervised learning
 - Is easy if you start from a good set of features
 - Otherwise you'll need some nonlinearity
- A shallow neural network takes care of both tasks at the same time
 - The first layer learns features
 - The last layer learns the regressor
- Universal Approximation Theorem
 - If there are many many hidden units, a shallow neural network can approximate any (nonlinear) function
 - e.g. Sinusoidal functions
 - But how many?



Feature Learning for a Shallow Network

- MNIST

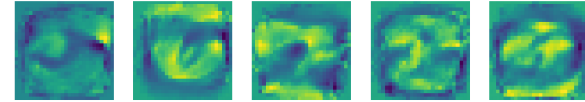
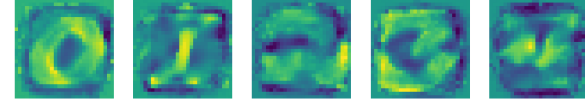
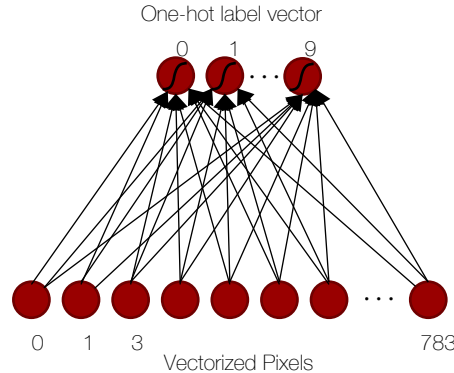
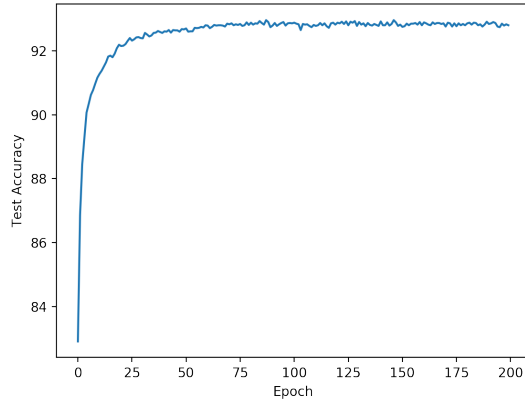
- MNIST Hand Written Digit Recognition
 - Considered as the “Hello, World!” problem in deep learning
 - $28 \times 28 = 784$ pixels per image
 - 55,000 training images
 - 10,000 testing images



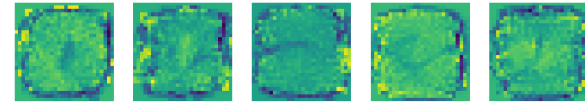
Feature Learning for a Shallow Network

- Softmax regression

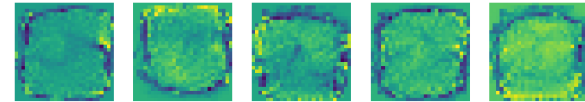
- A softmax classification with no feature extraction
 - i.e. Raw pixel intensities are the features



Weights at 20th epoch



- It's not bad, but how about some feature extraction?
 - YALT last time



Weights at 200th epoch

Feature Learning for a Shallow Network

- AEs for feature learning

- Let's extract some features, and then do the softmax
- Two cases: naïve AE versus sparse AE
- The AE objective function

$$\arg \min_{\mathbf{W}, \mathbf{H}} \mathcal{D}(\mathbf{X} || \mathbf{W}^\dagger \mathbf{H}) + \lambda g(\mathbf{H}), \quad \text{where } \mathbf{H} = \sigma(\mathbf{W} \mathbf{X})$$

- With the sparsity constraint

$$\hat{\alpha} = \frac{1}{T} \sum_{j=0}^{T-1} \mathbf{H}_{i,j}$$

$$g(\mathbf{H}_{i,:}) = KL(\alpha || \hat{\alpha}) = -\alpha \log \left(\frac{\alpha}{\hat{\alpha}} \right) - (1 - \alpha) \log \left(\frac{1 - \alpha}{1 - \hat{\alpha}} \right)$$

$$g(\mathbf{H}) = \sum_i g(\mathbf{H}_{i,:})$$

- For example, $x = [1, 0, 1, 0, 0, 0, 0, 0, 0, 0]$ $x = [0.8, 0, 0.9, 0, 0.2, 0, 0.1, 0, 0, 0]$

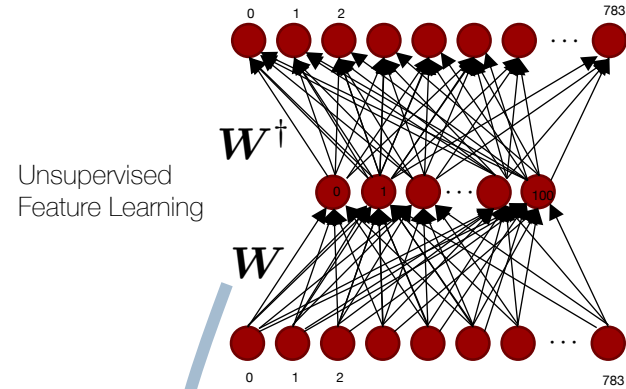
$$\hat{\alpha} = 0.2$$

$$\hat{\alpha} = 0.2$$

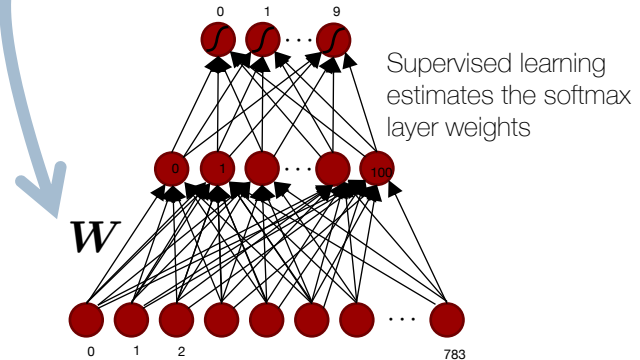
$$\text{If } \alpha = 0.2 \quad g(x) = 0$$

- A less sparse one $x = [0.8, 1, 0.9, 1, 0.2, 0, 0.1, 0, 0, 0]$ $g(x) = 0.092$

$$\hat{\alpha} = 0.4$$



Initialize the weight matrix with the learned one and don't update

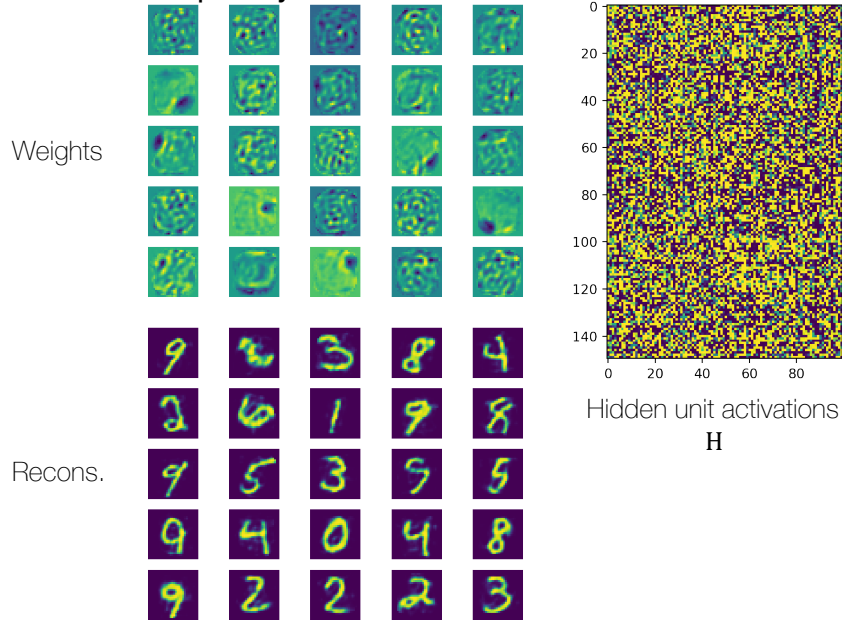


Feature Learning for a Shallow Network

- AEs for feature learning

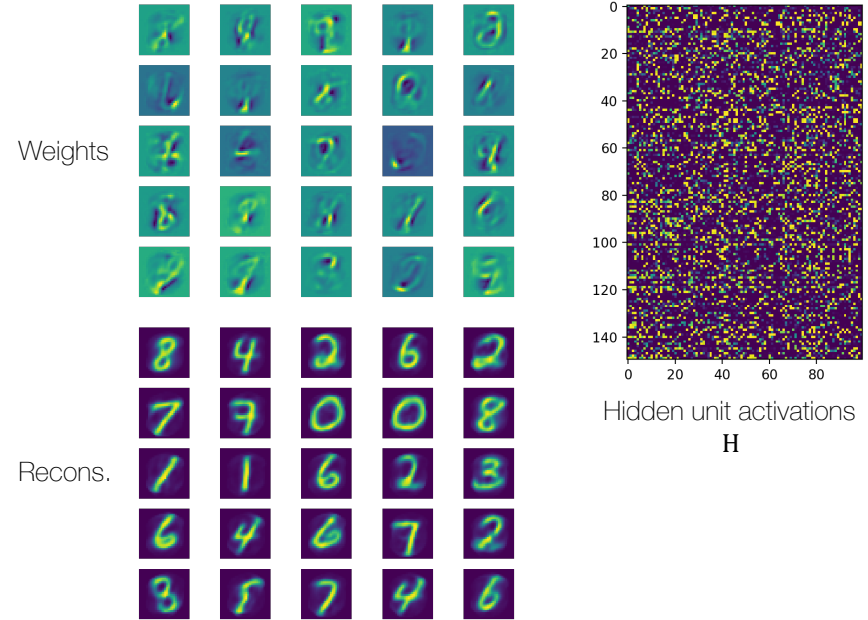
○ Case A: Naïve AE

□ No sparsity constraint



○ Case B: Sparse AE

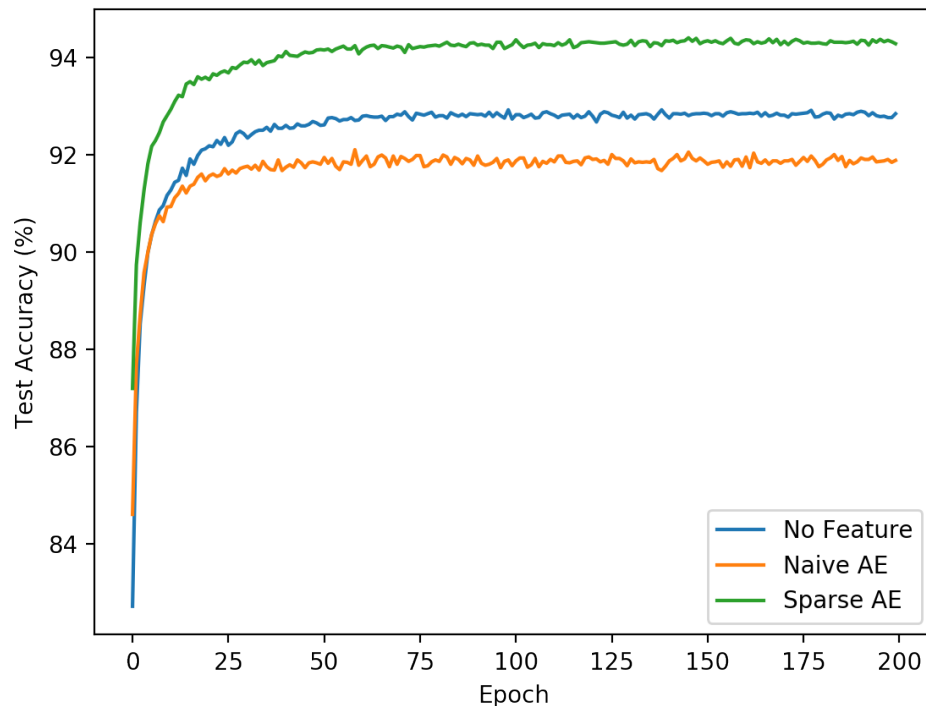
$$\bullet \lambda = 0.1, \alpha = 0.2$$



Feature Learning for a Shallow Network

- AEs for feature learning

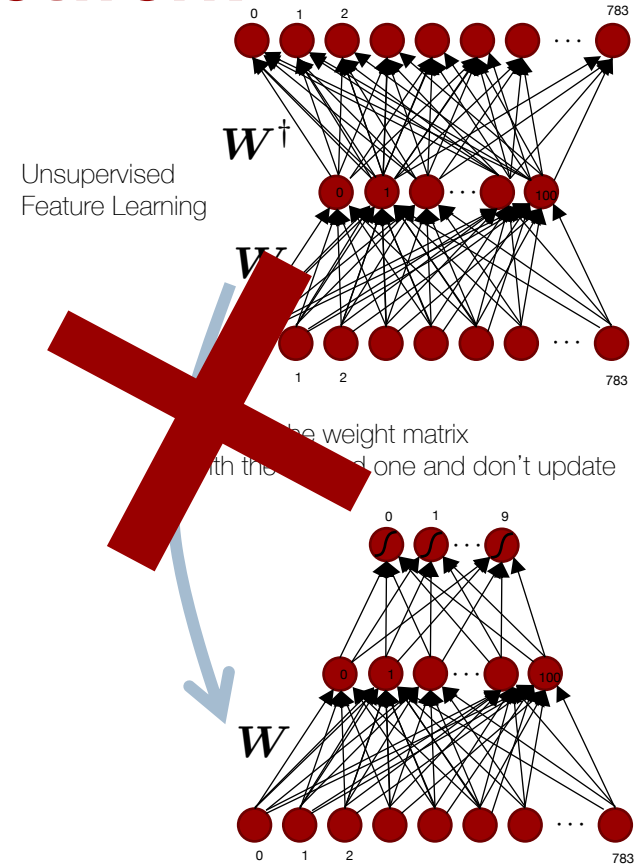
- A direct softmax classification on the raw pixel intensities (blue line)
 - Works pretty well
 - No features
- If your features are NOT good enough (naïve AE)
 - They degrade the performance
- If your features are good enough (sparse AE)
 - They improve the performance



Feature Learning for a Shallow Network

- Feature extraction + softmax?

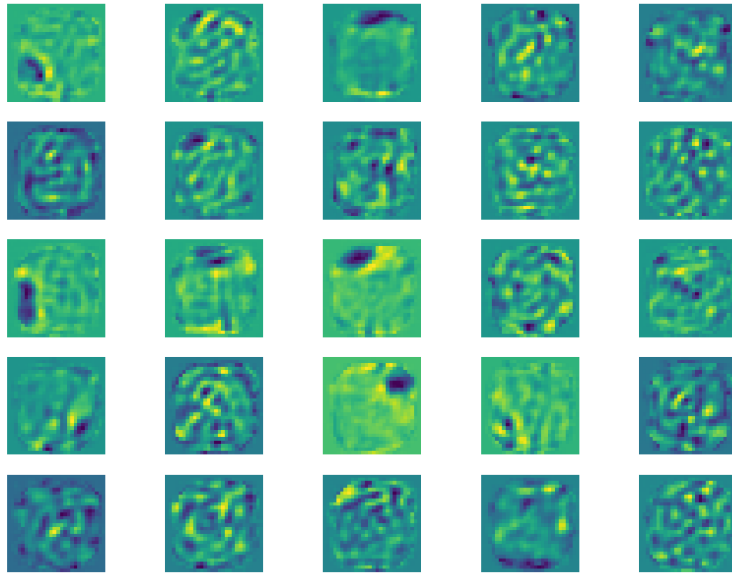
- Big question: why bother learning the features?
 - Feature learning can have a lot of different choices
 - e.g. regularization, constraints, ranks, error functions, activation functions, etc.
 - Could be unpredictable until you go ahead and test them out
 - Supervised learning has a lot of options, too
 - e.g. choice of the kernel functions, generative models, hyperparameters, etc.
- We can learn a neural network with one hidden layer
 - Randomly initialize both weight matrices in the two layers
 - Estimate them using backpropagation
 - Does it work better than the previous methods with feature engineering?
 - See the graph in the next page



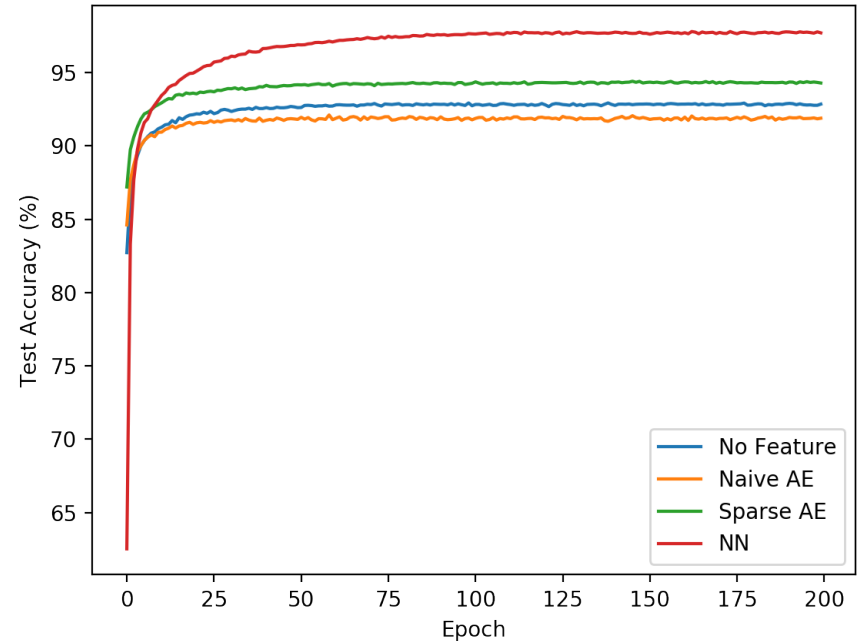
Backpropagation on a Shallow Network

- Feature learning is harmonized with NN training

- The first layer works as the feature learning process



The First Layer Weights



Backpropagation on a Shallow Network

- Going under the last layer

○ $i=0$

1. Initialize parameters with small random numbers
2. Calculate the output using all training samples (i.e. input and target pairs)

$$\mathbf{Z}^{(1)} \leftarrow \mathbf{W}^{(1)} \mathbf{X}^{(1)} + \mathbf{b}^{(1)} \quad \mathbf{X}^{(2)} \leftarrow \sigma(\mathbf{Z}^{(1)})$$

$$\mathbf{Z}^{(2)} \leftarrow \mathbf{W}^{(2)} \mathbf{X}^{(2)} + \mathbf{b}^{(2)} \quad \hat{\mathbf{Y}} \leftarrow g(\mathbf{Z}^{(2)}) \quad \text{Softmax for classification}$$

3. Calculate the error (cost)

$$\mathcal{E} = \sum_t \mathcal{D}(\mathbf{Y}_{:,t} || \hat{\mathbf{Y}}_{:,t}) \quad \text{Cross entropy}$$

4. Update the parameters

$$\mathbf{W}^{(2)} \leftarrow \mathbf{W}^{(2)} - \rho \frac{\partial \mathcal{E}}{\partial \mathbf{W}^{(2)}}$$

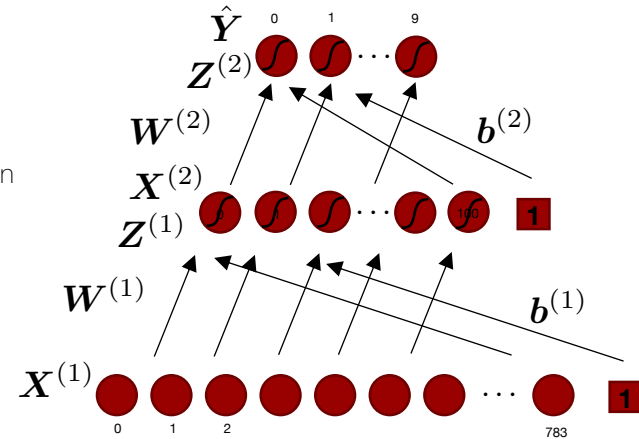
$$\mathbf{W}^{(1)} \leftarrow \mathbf{W}^{(1)} - \rho \frac{\partial \mathcal{E}}{\partial \mathbf{W}^{(1)}}$$

$$\mathbf{b}^{(2)} \leftarrow \mathbf{b}^{(2)} - \rho \frac{\partial \mathcal{E}}{\partial \mathbf{b}^{(2)}}$$

$$\mathbf{b}^{(1)} \leftarrow \mathbf{b}^{(1)} - \rho \frac{\partial \mathcal{E}}{\partial \mathbf{b}^{(1)}}$$

○ $i>0$

- Repeat 2-4



Backpropagation on a Shallow Network

- Going under the last layer

- For $W^{(2)}$ and $b^{(2)}$

- YALT

$$\frac{\partial \mathcal{E}}{\partial W^{(2)}} = \underbrace{\frac{\partial \mathcal{E}}{\partial \hat{Y}}}_{\text{BP Error } \delta^{(2)}} \underbrace{\frac{\partial \hat{Y}}{\partial Z^{(2)}}}_{\text{BP Error } \delta^{(2)}} \frac{\partial Z^{(2)}}{\partial W^{(2)}} = \underbrace{(\hat{Y} - Y)}_{\text{BP Error } \delta^{(2)}} X^{(2)\top} \quad \frac{\partial \mathcal{E}}{\partial b^{(2)}} = (\hat{Y} - Y) \mathbf{1}$$

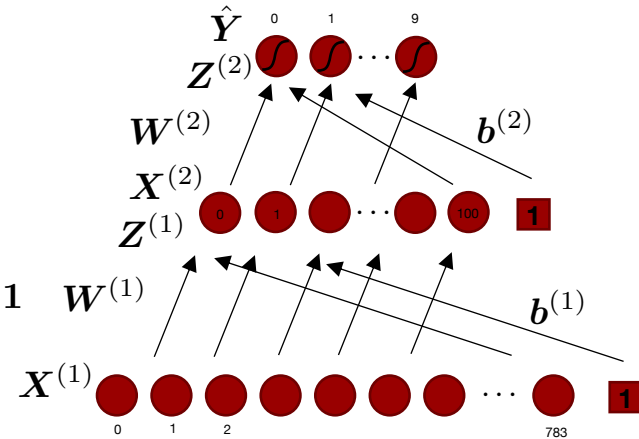
- Backpropagation Error $\delta^{(l)}$: the derivative of the cost function w.r.t. the **input** to the (l) -th hidden layer **units**

- For $W^{(1)}$ and $b^{(1)}$

$$\begin{aligned} \frac{\partial \mathcal{E}}{\partial W^{(1)}} &= \underbrace{\frac{\partial \mathcal{E}}{\partial \hat{Y}}}_{\text{BP Error } \delta^{(2)}} \underbrace{\frac{\partial \hat{Y}}{\partial Z^{(2)}}}_{\text{BP Error } \delta^{(1)}} \underbrace{\frac{\partial Z^{(2)}}{\partial X^{(2)}}}_{\text{BP Error } \delta^{(1)}} \underbrace{\frac{\partial X^{(2)}}{\partial Z^{(1)}}}_{\text{BP Error } \delta^{(1)}} \underbrace{\frac{\partial Z^{(1)}}{\partial W^{(1)}}}_{\text{BP Error } \delta^{(1)}} \\ &= \left(\underbrace{\left(W^{(2)\top} (\hat{Y} - Y) \right)}_{\text{BP Error } \delta^{(2)}} \odot \underbrace{\sigma'(Z^{(1)})}_{\text{BP Error } \delta^{(1)}} \right) X^{(1)\top} \end{aligned}$$

$$\frac{\partial \mathcal{E}}{\partial b^{(1)}} = \delta^{(1)} \mathbf{1}$$

$$\begin{aligned} Z^{(1)} &\leftarrow W^{(1)} X^{(1)} + b^{(1)} \\ X^{(2)} &\leftarrow \sigma(Z^{(1)}) \\ Z^{(2)} &\leftarrow W^{(2)} X^{(2)} + b^{(2)} \\ \hat{Y} &\leftarrow g(Z^{(2)}) \end{aligned}$$

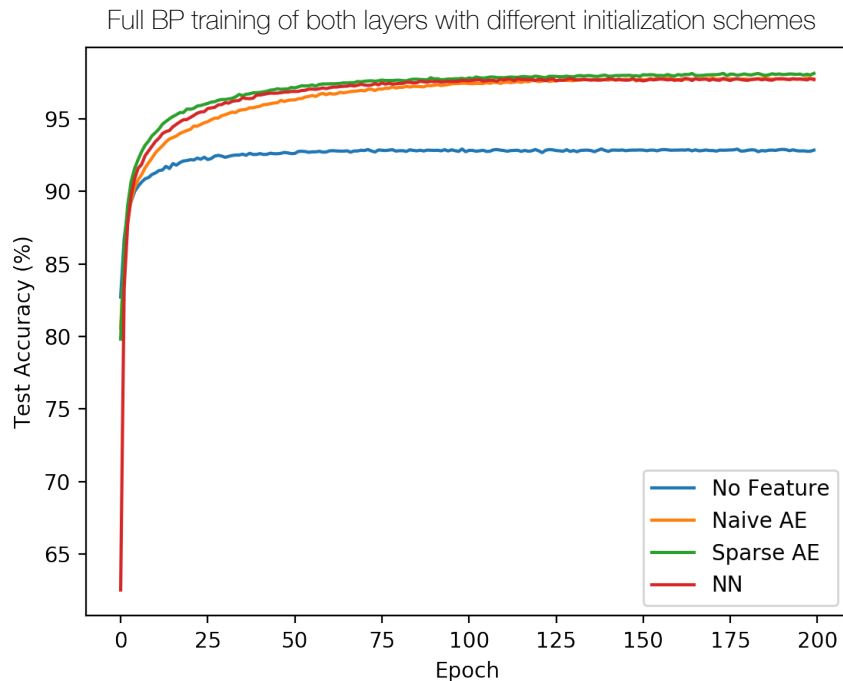


- The gradient for $W^{(l)}$ is [BP Error at (l)] X [Input from the previous layer]
- $\delta^{(l)}$: [BP Error at $(l + 1)$] X [Weights at $(l + 1)$] X [Derivative of the activation at (l)]

Vanishing Gradients in the Deep Neural Networks

- What happens if the network gets deeper?

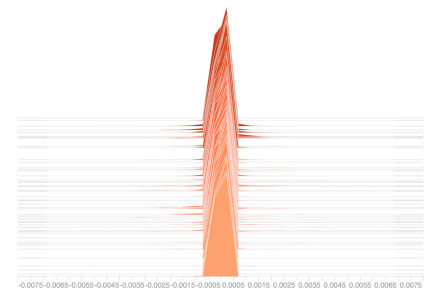
- We noticed that a separate feature learning procedure is NOT super helpful for softmax classification
 - A direct BP down to the first layer is doable and works better
 - Why is it better?
 - The feature learning part (i.e. the first layer weights) is aware of the discrimination objective
 - It might not be true if the last layer is more complicated (e.g. SVM)
 - Maybe the features don't really have to be that great
- Another option
 - What if we initialize the first layer with the trained AE weights, and then update weights of both layers?
 - See the graph
 - A better initialization doesn't seem to contribute to the convergence (why?)
 - BP just works down to the first layer
- Conclusion (so far): we don't need pretraining for shallow NNs
- What if the network is deeper?



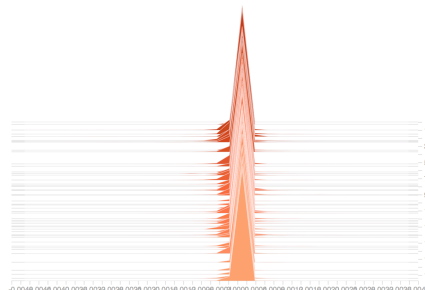
Vanishing Gradients in the Deep Neural Networks

- The vanishing gradient problem

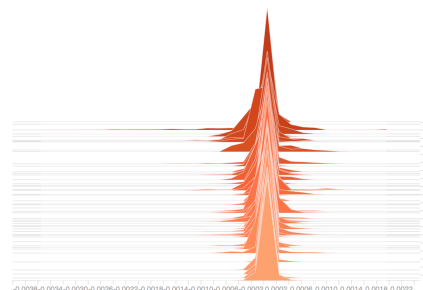
MNIST/1stLayer/W1_0-grad



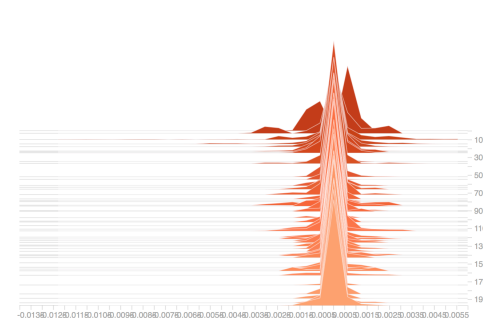
MNIST/2ndLayer/W2_0-grad



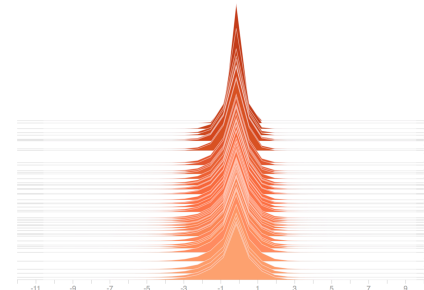
MNIST/3rdLayer/W3_0-grad



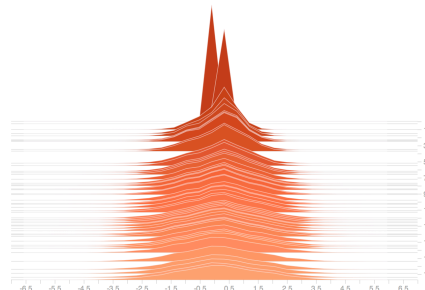
MNIST/4thLayer/W4_0-grad



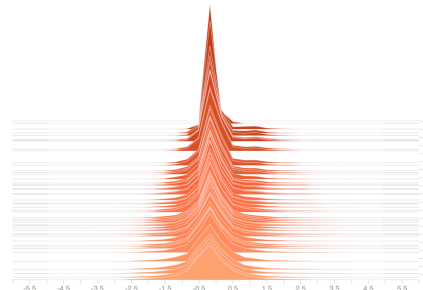
MNIST/1stLayer/W1_0-variable



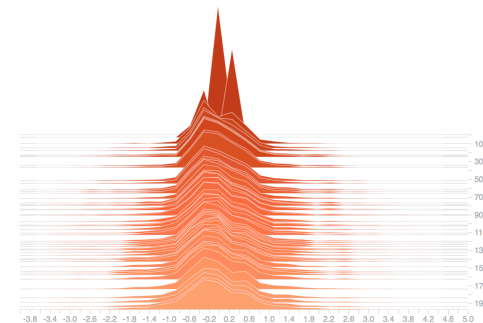
MNIST/2ndLayer/W2_0-variable



MNIST/3rdLayer/W3_0-variable



MNIST/4thLayer/W4_0-variable



Vanishing Gradients in the Deep Neural Networks

- Speed bumps

- It's due to the sigmoid function. What's wrong with them?

- Their gradients are small when the function is saturated

- Suppose a DNN with L hidden layers

$$\mathbf{Z}^{(1)} \leftarrow \mathbf{W}^{(1)} \mathbf{X}^{(1)} + \mathbf{b}^{(1)}$$

$$\mathbf{X}^{(2)} \leftarrow \sigma(\mathbf{Z}^{(1)})$$

$$\mathbf{Z}^{(2)} \leftarrow \mathbf{W}^{(2)} \mathbf{X}^{(2)} + \mathbf{b}^{(2)}$$

$$\mathbf{X}^{(3)} \leftarrow \sigma(\mathbf{Z}^{(2)})$$

...

$$\mathbf{Z}^{(L)} \leftarrow \mathbf{W}^{(L)} \mathbf{X}^{(L)} + \mathbf{b}^{(L)}$$

$$\mathbf{X}^{(L+1)} \leftarrow \sigma(\mathbf{Z}^{(L)})$$

$$\mathbf{Z}^{(L+1)} \leftarrow \mathbf{W}^{(L+1)} \mathbf{X}^{(L+1)} + \mathbf{b}^{(L+1)}$$

$$\hat{\mathbf{Y}} \leftarrow \text{softmax}(\mathbf{Z}^{(L+1)})$$

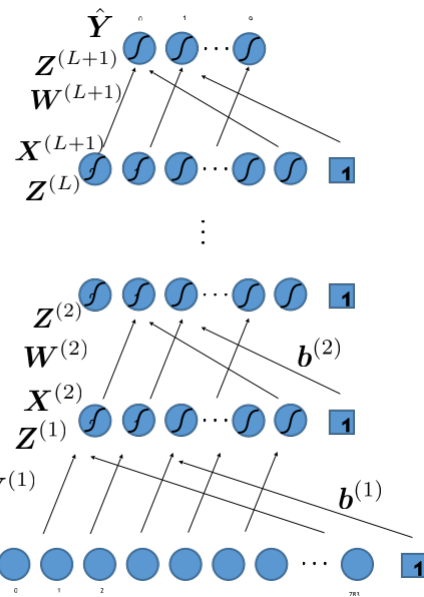
- Gradients per layer

$$\frac{\partial \mathcal{E}}{\partial \mathbf{W}^{(L+1)}} = \underbrace{\frac{\partial \mathcal{E}}{\partial \hat{\mathbf{Y}}} \frac{\partial \hat{\mathbf{Y}}}{\partial \mathbf{Z}^{(L+1)}}}_{\text{BP Error } \delta^{(L+1)}} \frac{\partial \mathbf{Z}^{(L+1)}}{\partial \mathbf{W}^{(L+1)}} = (\hat{\mathbf{Y}} - \mathbf{Y}) \mathbf{X}^{(L+1)\top}$$

$$\frac{\partial \mathcal{E}}{\partial \mathbf{W}^{(L)}} = \underbrace{\frac{\partial \mathcal{E}}{\partial \hat{\mathbf{Y}}} \frac{\partial \hat{\mathbf{Y}}}{\partial \mathbf{Z}^{(L+1)}} \frac{\partial \mathbf{Z}^{(L+1)}}{\partial \mathbf{X}^{(L+1)}} \frac{\partial \mathbf{X}^{(L+1)}}{\partial \mathbf{Z}^{(L)}} \frac{\partial \mathbf{Z}^{(L)}}{\partial \mathbf{W}^{(L)}}}_{\delta^{(L)}} = \left((\mathbf{W}^{(L+1)\top} (\hat{\mathbf{Y}} - \mathbf{Y})) \odot \sigma'(\mathbf{Z}^{(L)}) \right) \mathbf{X}^{(L)\top}$$

$$\frac{\partial \mathcal{E}}{\partial \mathbf{W}^{(1)}} = \frac{\partial \mathcal{E}}{\partial \hat{\mathbf{Y}}} \frac{\partial \hat{\mathbf{Y}}}{\partial \mathbf{Z}^{(L+1)}} \prod_{l=1}^L \left[\frac{\partial \mathbf{Z}^{(l+1)}}{\partial \mathbf{X}^{(l+1)}} \frac{\partial \mathbf{X}^{(l+1)}}{\partial \mathbf{Z}^{(l)}} \right] \frac{\partial \mathbf{Z}^{(1)}}{\partial \mathbf{W}^{(1)}} = \left(\left(\mathbf{W}^{(2)\top} (\dots (\mathbf{W}^{(L+1)\top} (\hat{\mathbf{Y}} - \mathbf{Y})) \odot \sigma'(\mathbf{Z}^{(L)}) \dots) \odot \sigma'(\mathbf{Z}^{(1)}) \right) \mathbf{X}^{(1)\top} \right)_{\delta^{(1)}}$$

L speed bumps



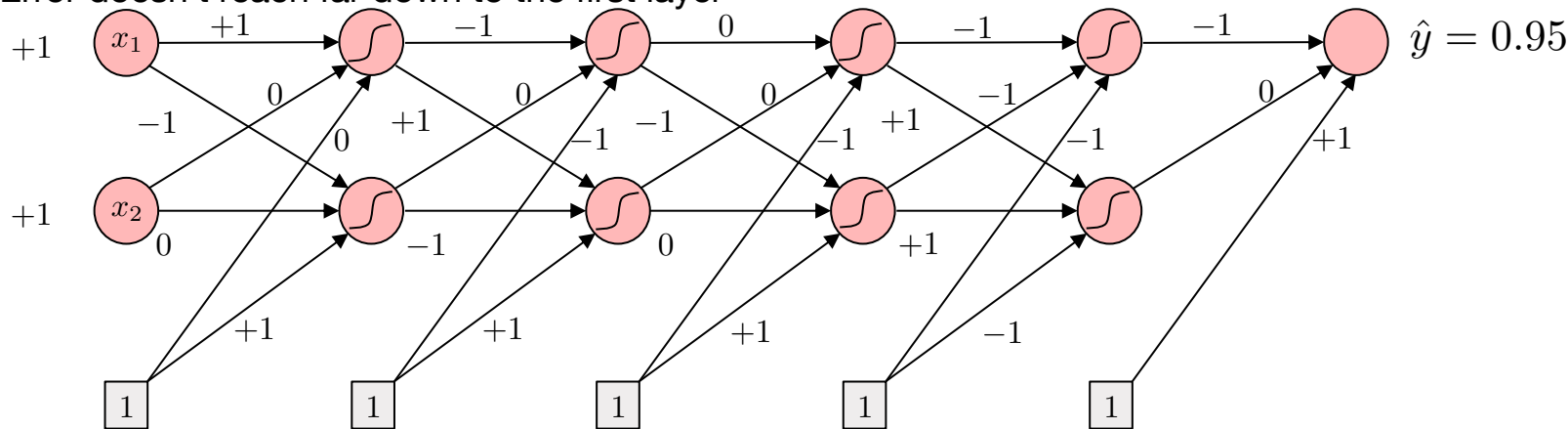
INDIANA UNIVERSITY

SCHOOL OF INFORMATICS, COMPUTING, AND ENGINEERING

Vanishing Gradients in the Deep Neural Networks

- Let's see what happens in a small network

- Error doesn't reach far down to the first layer



$$\begin{aligned}\delta^{(1)} &= \left(\mathbf{A}^{(2)\top} \delta^{(2)} \right) \odot g'(\mathbf{z}^{(1)}) \\ &= \begin{bmatrix} 0.0005 \\ 0 \end{bmatrix} \odot \begin{bmatrix} 0.42 \\ 1 \end{bmatrix} = \begin{bmatrix} \underline{0.0002} \\ \underline{0} \end{bmatrix}\end{aligned}$$

$$\begin{aligned}\delta^{(3)} &= \left(\mathbf{A}^{(4)\top} \delta^{(4)} \right) \odot g'(\mathbf{z}^{(3)}) \\ &= \begin{bmatrix} 0.058 \\ 0.058 \end{bmatrix} \odot \begin{bmatrix} 0.42 \\ 0.0789 \end{bmatrix} = \begin{bmatrix} \underline{0.0244} \\ \underline{0.0046} \end{bmatrix}\end{aligned}$$

$$\begin{aligned}\hat{y} - y &= 0.95 - (-1) \\ \delta^{(5)} &= (\hat{y} - y) \sigma'(z^{(5)}) \\ &= \underline{1.95} \cdot 0.0973 = \underline{0.1897}\end{aligned}$$

$$\begin{aligned}\delta^{(2)} &= \left(\mathbf{A}^{(3)\top} \delta^{(3)} \right) \odot g'(\mathbf{z}^{(2)}) \\ &= \begin{bmatrix} -0.0046 \\ 0 \end{bmatrix} \odot \begin{bmatrix} 0.1114 \\ 0.1114 \end{bmatrix} = \begin{bmatrix} \underline{-0.0005} \\ \underline{0} \end{bmatrix}\end{aligned}$$

$$\begin{aligned}\delta^{(4)} &= \left(\mathbf{A}^{(5)\top} \delta^{(5)} \right) \odot g'(\mathbf{z}^{(4)}) \\ &= \begin{bmatrix} -0.1897 \\ 0 \end{bmatrix} \odot \begin{bmatrix} 0.3060 \\ 0.5577 \end{bmatrix} = \begin{bmatrix} \underline{-0.0580} \\ \underline{0} \end{bmatrix}\end{aligned}$$



Greedy Layer-Wise Unsupervised Learning

- Pretraining the network

- If you backpropagate the error from the final layer
 - The error vanishes (most of the time) in the first layer
 - So, you can't really update the first layer weights
- The first (historical) way to overcome this problem:
 - Stack up the greedy layer-wise unsupervised learning results
 - Initialize the network parameters with them
 - Then, the first layer weights can start from a very good initialization
- Greedy layer-wise unsupervised learning?
 - You've learned a lot of them
 - PCA, ICA, NMF, GMM, etc
 - So, what do I mean by “staking up” them?



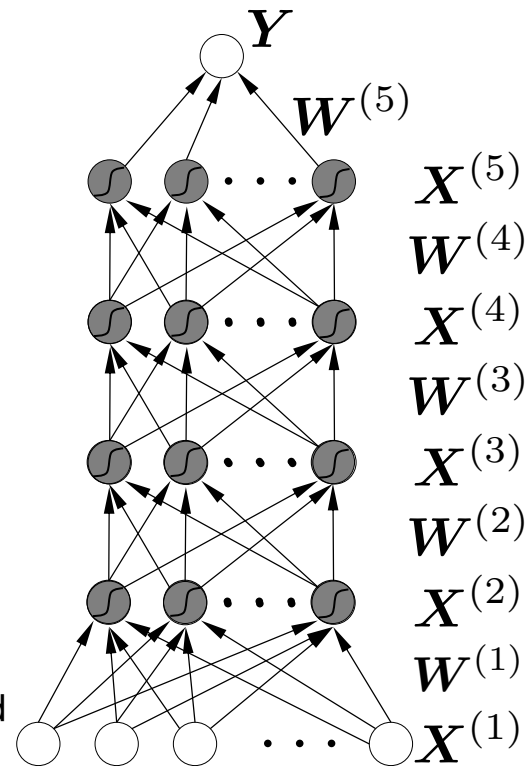
Greedy Layer-Wise Unsupervised Learning

- A primitive method to train a DNN
 - The BP error vanishes in the lower layer due to the use of sigmoid functions
 - Pretraining learns the lower layer weights in a different way
- Stacked AE
 - For the 1st layer $\hat{\mathbf{X}}^{(1)} \approx \mathbf{W}^{(1)\dagger} \sigma(\mathbf{W}^{(1)} \mathbf{X}^{(1)})$
 - For the 2nd layer $\mathbf{X}^{(2)} \leftarrow \sigma(\mathbf{W}^{(1)} \mathbf{X}^{(1)})$
 $\hat{\mathbf{X}}^{(2)} \approx \mathbf{W}^{(2)\dagger} \sigma(\mathbf{W}^{(2)} \mathbf{X}^{(2)})$
 - For the (l)-th layer $\mathbf{X}^{(l)} \leftarrow \sigma(\mathbf{W}^{(l-1)} \mathbf{X}^{(l-1)})$
 $\hat{\mathbf{X}}^{(l)} \approx \mathbf{W}^{(l)\dagger} \sigma(\mathbf{W}^{(l)} \mathbf{X}^{(l)})$
- What kinds of autoencoders are there?
 - Hinton's first pretraining method used Restricted Boltzman Machine
 - Andrew Ng's group showed sparse AEs work well
 - Bengio's group suggested denoising AEs

Greedy Layer-Wise Unsupervised Learning

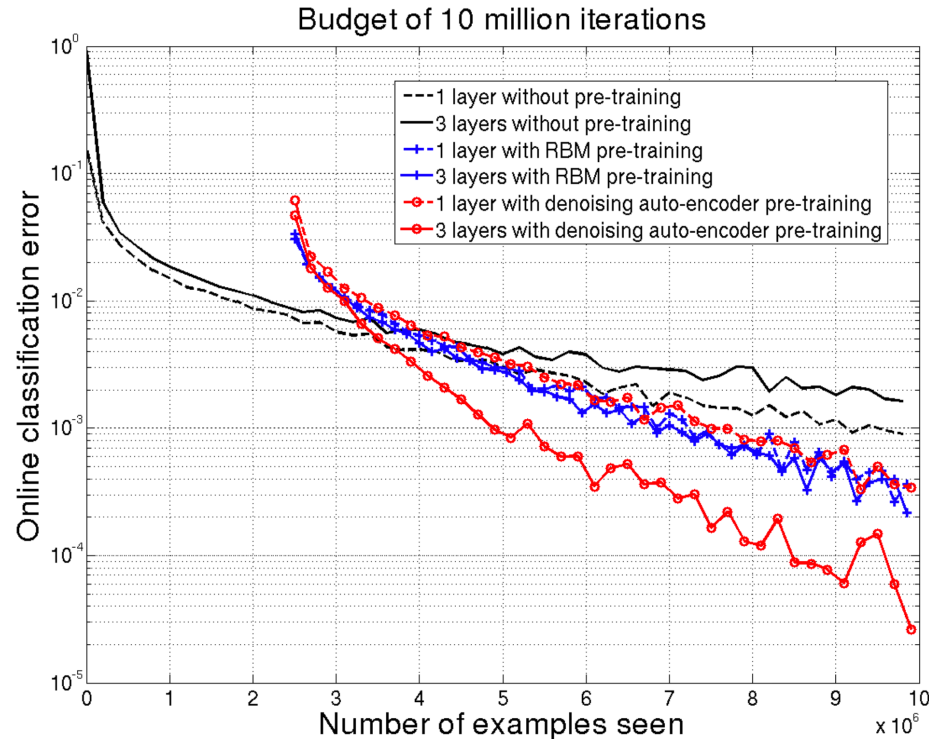
- Finetuning

- Now that you know some better weights, the DNN can be initialized with the pretrained weights
 - Any problems with this?
- Each layer-wise AE creates approximation error
 - which propagates up to the last layer
 - Last layer weights in the stacked AE are not in good quality
 - What can we do?
- Finetuning
 - This procedure is not different from a regular BP
 - It might not be very effective to update the lower layers
 - But can fix the last layer weights
 - A good solution for the stacked AE
- The pretraining-and-finetuning pipeline was the first deep learning method
- But the pretraining technique is known to be difficult to handle



Greedy Layer-Wise Unsupervised Learning

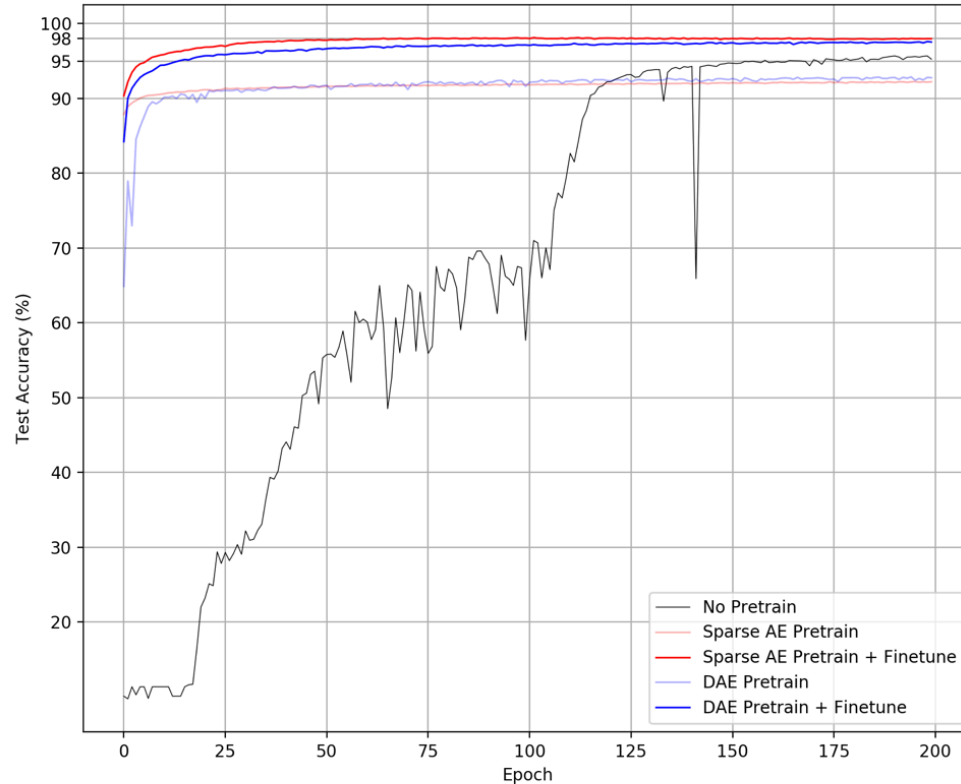
- Some convincing results on MNIST



Greedy Layer-Wise Unsupervised Learning

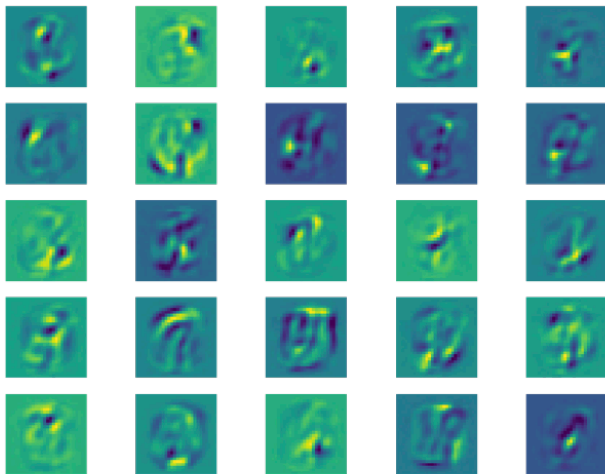
- Pretraining and finetuning

- 100X3 logistic hidden units
- Stochastic gradient descent

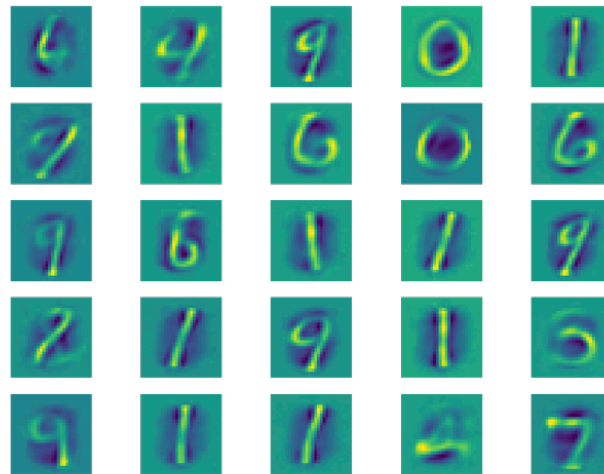


Take-Home Messages

- The thing is, training such autoencoders is tricky in a lot of senses
 - There are some criteria known to work (e.g. sparsity), but it's difficult to know until we try them out
 - It's unsupervised learning, so there's a chance of overfitting
 - Some hyperparameters to tune as usual
 - e.g. What would be the right amount of sparsity?



$\alpha = 0.5$



$\alpha = 0.05$

Take-Home Messages

- More importantly, the higher layers are hard to visualize to figure out the best way to train
- What would be the other ways to train DNNs?
 - Choosing the activation function wisely
 - Securing the pathway of BP error
 - Better optimization methods help, too



Reading

- Hinton, Geoffrey E., Simon Osindero, and Yee-Whye Teh. "A fast learning algorithm for deep belief nets." Neural computation 18.7 (2006): 1527-1554.
- Lee, Honglak, Chaitanya Ekanadham, and Andrew Y. Ng. "Sparse deep belief net model for visual area V2." Advances in neural information processing systems. 2008.
- Vincent, Pascal, et al. "Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion." Journal of Machine Learning Research 11.Dec (2010): 3371-3408.





Thank You!

